

MÃO NA MASSA

Aqui está o resultado final consolidado do exercício "**Mão na Massa**", organizado para que possas copiar, testar e utilizar como base para o teu projeto da Situação de Aprendizagem (SA).

O exercício está dividido em dois arquivos: a **Classe (o molde)** e o **Script de Teste (a execução)**.

1. O arquivo da Classe: Produto.php

Este arquivo contém toda a lógica de negócio, proteção de dados (encapsulamento) e formatação.

PHP

<?php

```
class Produto {
    // Atributos privados: ninguém acede diretamente de fora da classe
    private string $nome;
    private float $preco;
    private int $estoque;

    /**
     * Passo 2: Construtor
     * Inicializa o objeto e já aplica a validação de estoque
     */
    public function __construct(string $nome, float $preco, int $estoque) {
        $this->nome = $nome;
        $this->preco = $preco;
        $this->setEstoque($estoque);
    }

    /**
     * Passo 3: Getters e Setters (Acesso Seguro)
     */

    // Recupera o nome
    public function getNome(): string {
        return $this->nome;
    }

    // Recupera o stock
    public function getEstoque(): int {
        return $this->estoque;
    }

    // Define o stock com regra de validação
    public function setEstoque(int $quantidade): void {
        if ($quantidade >= 0) {
            $this->estoque = $quantidade;
        } else {
            echo "<br>❌ <b>Erro de Validação:</b> A quantidade ($quantidade) não pode ser negativa.<br>";
        }
    }
}
```

```

    }

    /**
     * Método de Formatação
     * Transforma o valor numérico em moeda formatada (Ex: R$ 1.200,50)
     */
    public function exibirPrecoFormatado(): string {
        return "R$ " . number_format($this->preco, 2, ',', '.');
    }
}

```

2. O Script de Teste: index.php

Este arquivo é responsável por "dar vida" ao objeto e provar que as regras de segurança funcionam.

```

PHP
<?php
// Importa a definição da classe
require_once "Produto.php";

echo "<h1>🛒 Teste da Camada de Modelo</h1>";

// 1. Instanciando um produto com sucesso
$p1 = new Produto("Teclado Mecânico RGB", 250.00, 15);

echo "<b>Produto criado:</b> " . $p1->getNome() . "<br>";
echo "<b>Preço formatado:</b> " . $p1->exibirPrecoFormatado() . "<br>";
echo "<b>Estoque atual:</b> " . $p1->getEstoque() . " unidades.<br>";

echo "<hr>";

// 2. Testando a proteção de dados (Tentativa de erro)
echo "<b>Ação:</b> Tentando alterar stock para -5...<br>";
$p1->setEstoque(-5);

echo "<br><b>Verificação final de stock:</b> " . $p1->getEstoque() . " unidades.";
echo "<br><i>(O valor permaneceu 15? Se sim, a tua classe está protegida!)</i>";

```

Resumo do que foi construído:

- Proteção (Encapsulamento):** Ao tentar definir um stock negativo, o método `setEstoque` bloqueou a operação e exibiu uma mensagem de erro, mantendo a integridade do objeto.
- Padronização:** O método `exibirPrecoFormatado` garante que o preço será exibido da mesma forma em qualquer parte do sistema.
- Tipagem:** O uso de `: void`, `: string` e `: int` garante que o PHP te avise caso tentas retornar um tipo de dado errado, facilitando a manutenção futura.

Como rodar: Coloca ambos os arquivos na mesma pasta do seu servidor local (XAMPP, Wamp, Laragon) e acesse o `index.php` pelo navegador.

EXERCÍCIOS DE FIXAÇÃO

Aqui está o gabarito completo e comentado dos exercícios progressivos, utilizando as práticas recomendadas de PHP e Orientação a Objetos.

Nível 1: Arrays e Coleções

1. Arrays Indexados

PHP

```
$meses = ["Janeiro", "Fevereiro", "Março", "Abril", "Maio", "Junho", "Julho", "Agosto", "Setembro", "Outubro", "Novembro", "Dezembro"];  
echo $meses[5]; // Saída: Junho (índice 0 é Janeiro, 5 é Junho)
```

2. Manipulação Simples

PHP

```
$cidades = ["Joinville", "Curitiba"];  
$cidades[] = "Florianópolis"; // Adiciona ao final
```

3. Array Associativo

PHP

```
$perfil = ["nome" => "João", "idade" => 25, "cidade" => "Brusque"];  
echo "Olá, meu nome é {$perfil['nome']} e moro em {$perfil['cidade']}.";
```

4. Alteração de Valores

PHP

```
$perfil['idade'] = 26;
```

5. Cuidado com as Aspas

PHP

```
$config['versao'] = 1.2; // Correção: adicionadas aspas na chave 'versao'
```

6. Case Sensitivity

- **Resposta:** O PHP não encontraria a chave. Ele emitiria um erro do tipo *Warning: Undefined array key "Nome"*, pois para o PHP, "nome" e "Nome" são identificadores diferentes.

7. Multidimensional Simples

PHP

```
$estoque = [  
    ["nome" => "Teclado", "quantidade" => 10],  
    ["nome" => "Mouse", "quantidade" => 25]  
];
```

8. Acesso em Matriz

PHP

```
echo $estoque[1]['quantidade']; // Acessa o índice 1 (segundo item) e a chave 'quantidade'
```

9. Estrutura de Tabela

PHP

```
$usuarios = [  
    ["login" => "admin", "perfil" => "Administrador"],  
    ["login" => "user1", "perfil" => "Editor"],  
    ["login" => "guest", "perfil" => "Leitor"]  
];
```

10. Inserção Dinâmica

PHP

```
$usuarios[] = ["login" => "novo_user", "perfil" => "Editor"];
```

Nível 2: Pilares da POO - Classes e Atributos

11. Definição de Molde

PHP

```
class Veiculo { }
```

12. Criação de Atributos

PHP

```
class Veiculo {  
    public $marca;  
    public $modelo;  
    public $ano;  
}
```

13. Instanciação

PHP

```
$meuCarro = new Veiculo;
```

14. Atribuição de Dados

PHP

```
$meuCarro->marca = "Toyota";  
$meuCarro->modelo = "Corolla";  
$meuCarro->ano = 2024;
```

15. Independência de Objetos

PHP

```
$carroTrabalho = new Veiculo;  
$carroTrabalho->marca = "Fiat";  
$carroTrabalho->modelo = "Uno";  
// Explicação: Cada objeto ocupa um espaço diferente na memória (instâncias distintas).
```

16. Visibilidade

- **Resposta:** O PHP gera um erro fatal (*Fatal error: Cannot access private property*). Atributos privados só podem ser lidos ou alterados por métodos dentro da própria classe.

Nível 3: Pilares da POO - Métodos e Lógica

17. Criação de Comportamento e 18. Uso do \$this

PHP

```
public function exibirDetalhes() {  
    return "Veículo: {$this->marca} {$this->modelo}.";  
}
```

18.

19. Método Construtor

PHP

```
public function __construct($marca, $modelo) {  
    $this->marca = $marca;  
    $this->modelo = $modelo;  
}  
  
// Uso: $carro = new Veiculo("Ford", "Ka");
```

20. Lógica de Proteção (Encapsulamento)

PHP

```
public function setAno($ano) {  
    if ($ano > 1886) {  
        $this->ano = $ano;  
    } else {  
        echo "Ano inválido!";  
    }  
}
```

Desafio Integrador (Conexão com a SA)

PHP

```
<?php  
class Produto {  
    public $nome;  
    public $preco;  
  
    public function __construct($nome, $preco) {  
        $this->nome = $nome;  
        $this->preco = $preco;  
    }  
}
```

// Array Multidimensional armazenando Objetos

```
$listaProdutos = [  
    new Produto("Monitor", 1200.00),  
    new Produto("Gabinete", 450.00),  
    new Produto("Memória RAM", 300.00)  
];
```

```
// Para acessar o nome do primeiro produto:
```

```
echo $listaProdutos[0]->nome;
```

```
?>
```